

Red Black Tree

Introduction:

A red-black tree is a kind of self-balancing binary search tree where each node has an extra bit, and that bit is often interpreted as the colour (red or black). These colours are used to ensure that the tree remains balanced during insertions and deletions. Although the balance of the tree is not perfect, it is good enough to reduce the searching time and maintain it around $O(\log n)$ time, where n is the total number of elements in the tree. This tree was invented in 1972 by Rudolf Bayer.

It must be noted that as each node requires only 1 bit of space to store the colour information, these types of trees show identical memory footprint to the classic (uncoloured) binary search tree.

Rules That Every Red-Black Tree Follows:

1. Every node has a colour either red or black.
2. The root of the tree is always black.
3. There are no two adjacent red nodes (A red node cannot have a red parent or red child).
4. Every path from a node (including root) to any of its descendants NULL nodes has the same number of black nodes.

Most of the BST operations (e.g., search, max, min, insert, delete.. etc) take $O(h)$ time where h is the height of the BST. The cost of these operations may become $O(n)$ for a skewed Binary tree. If we make sure that the height of the tree remains $O(\log n)$ after every insertion and deletion, then we can guarantee an upper bound of $O(\log n)$ for all these operations. The height of a Red-Black tree is always $O(\log n)$ where n is the number of nodes in the tree.

Comparison with AVL Tree:

The AVL trees are more balanced compared to Red-Black Trees, but they may cause more rotations during insertion and deletion. So if your application involves frequent insertions and deletions, then Red-Black trees should be preferred. And if the insertions and deletions are less frequent and search is a more frequent operation, then AVL tree should be preferred over Red-Black Tree.

Interesting points about Red-Black Tree:

1. Black height of the red-black tree is the number of black nodes on a path from the root node to a leaf node. Leaf nodes are also counted as black nodes. So, a red-black tree of height h has black height $\geq h/2$.
2. Height of a red-black tree with n nodes is $h \leq 2 \log_2(n + 1)$.

3. All leaves (NIL) are black.
4. The black depth of a node is defined as the number of black nodes from the root to that node i.e the number of black ancestors.
5. Every red-black tree is a special case of a binary tree.

Search Operation in Red-black Tree:

As every red-black tree is a special case of a binary tree so the searching algorithm of a red-black tree is similar to that of a binary tree.

searchElement (tree, val)

Step 1:

If tree -> data = val OR tree = NULL

Return tree

Else

If val < data

Return searchElement (tree -> left, val)

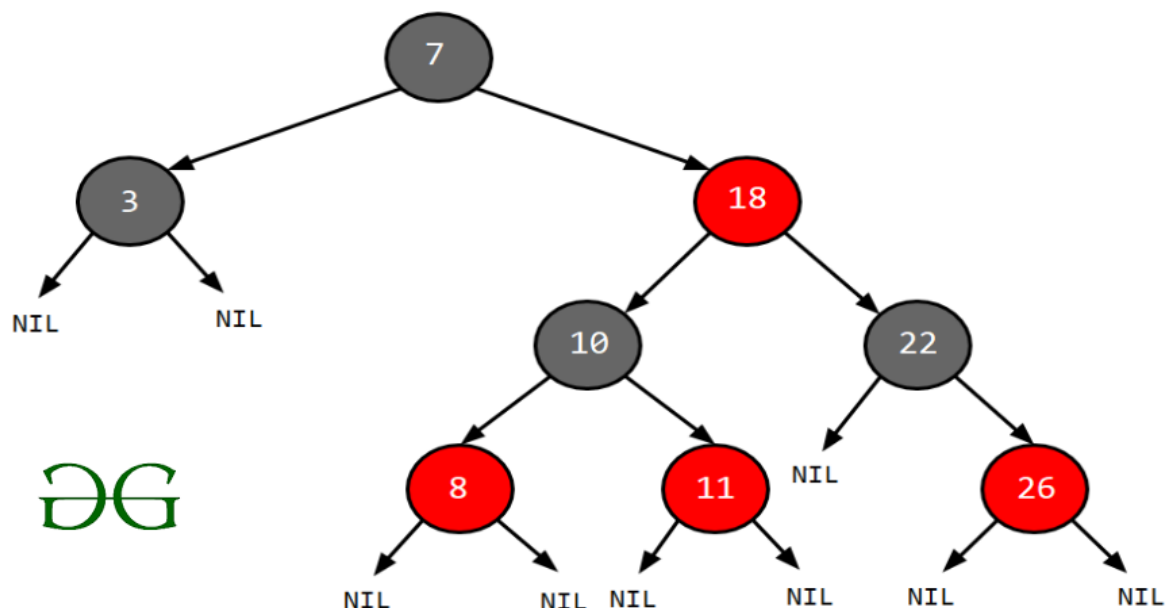
Else

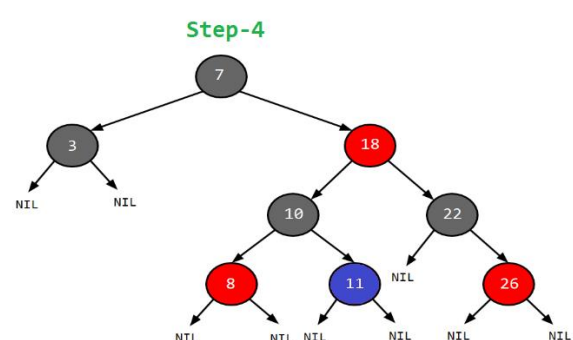
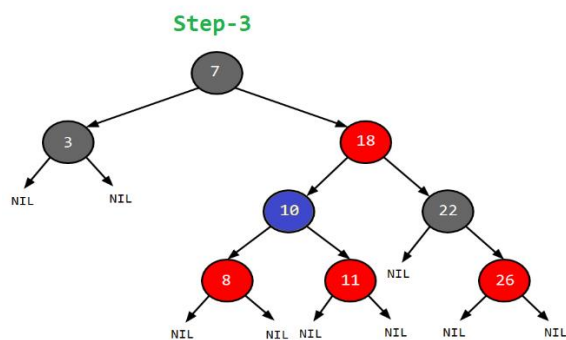
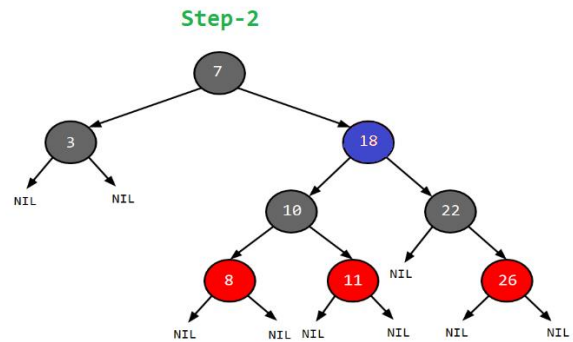
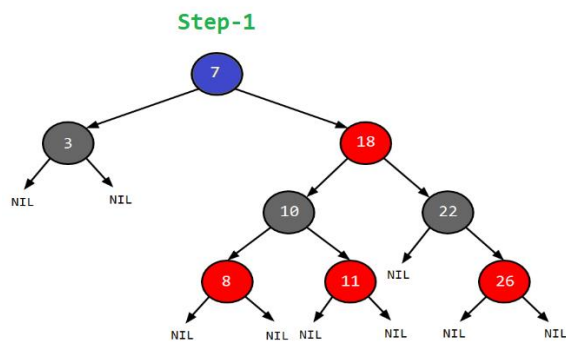
Return searchElement (tree -> right, val)

[End of if]

[End of if]

Step 2: END





Applications:

1. Most of the self-balancing BST library functions like map and set in C++ (OR TreeSet and TreeMap in Java) use Red-Black Tree.
2. It is used to implement CPU Scheduling Linux. Completely Fair Scheduler uses it.
3. Besides they are used in the K-mean clustering algorithm for reducing time complexity.
4. Moreover, MySQL also uses the Red-Black tree for indexes on tables.

Insertion in Red Black Tree:

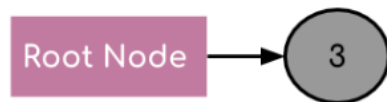
Algorithm:

Let x be the newly inserted node.

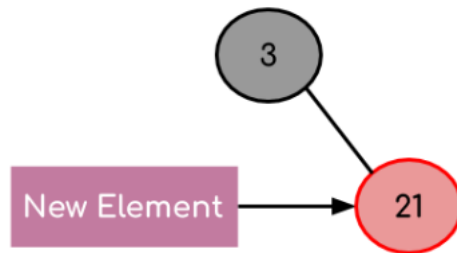
1. Perform standard BST insertion and make the colour of newly inserted nodes as RED.
2. If x is the root, change the colour of x as BLACK (Black height of complete tree increases by 1).
3. Do the following if the color of x's parent is not BLACK **and** x is not the root.
 - a) If x's uncle is RED (Grandparent must have been black from property
 - 4). (i) Change the colour of parent and uncle as BLACK.

- (ii) Colour of a grandparent as RED.
- (iii) Change $x = x$'s grandparent, repeat steps 2 and 3 for new x .
- b) If x 's uncle is BLACK**, then there can be four configurations for x , x 's parent (**p**) and x 's grandparent (**g**) (This is similar to AVL Tree)
 - (i) Left Left Case (p is left child of g and x is left child of p)
 - (ii) Left Right Case (p is left child of g and x is the right child of p)
 - (iii) Right Right Case (Mirror of case i)
 - (iv) Right Left Case (Mirror of case ii)

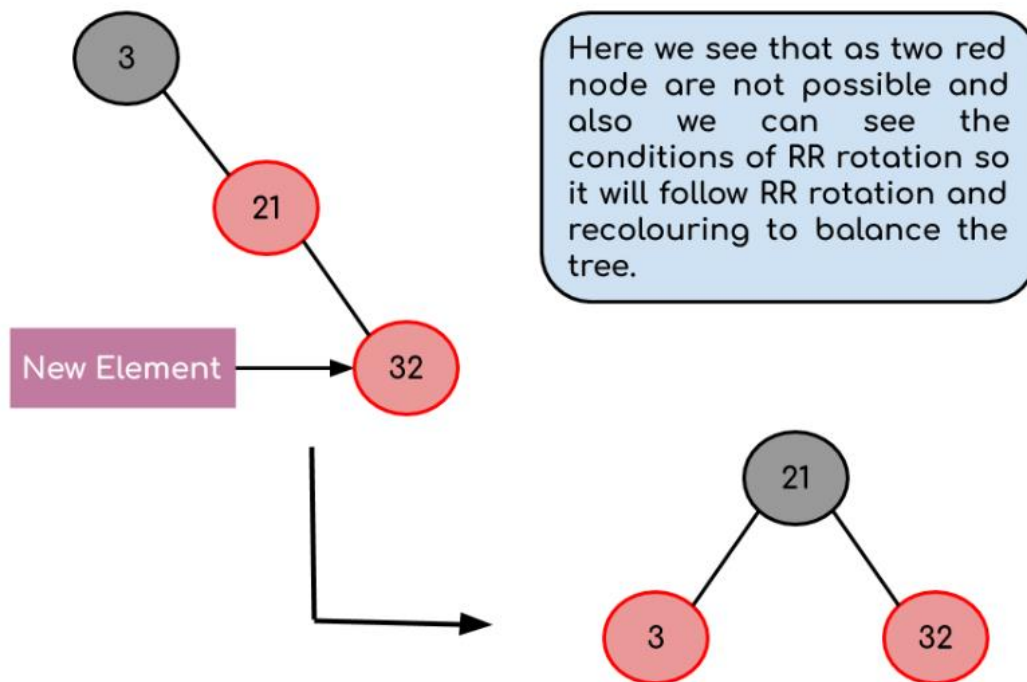
Step 1: Inserting element 3 inside the tree.



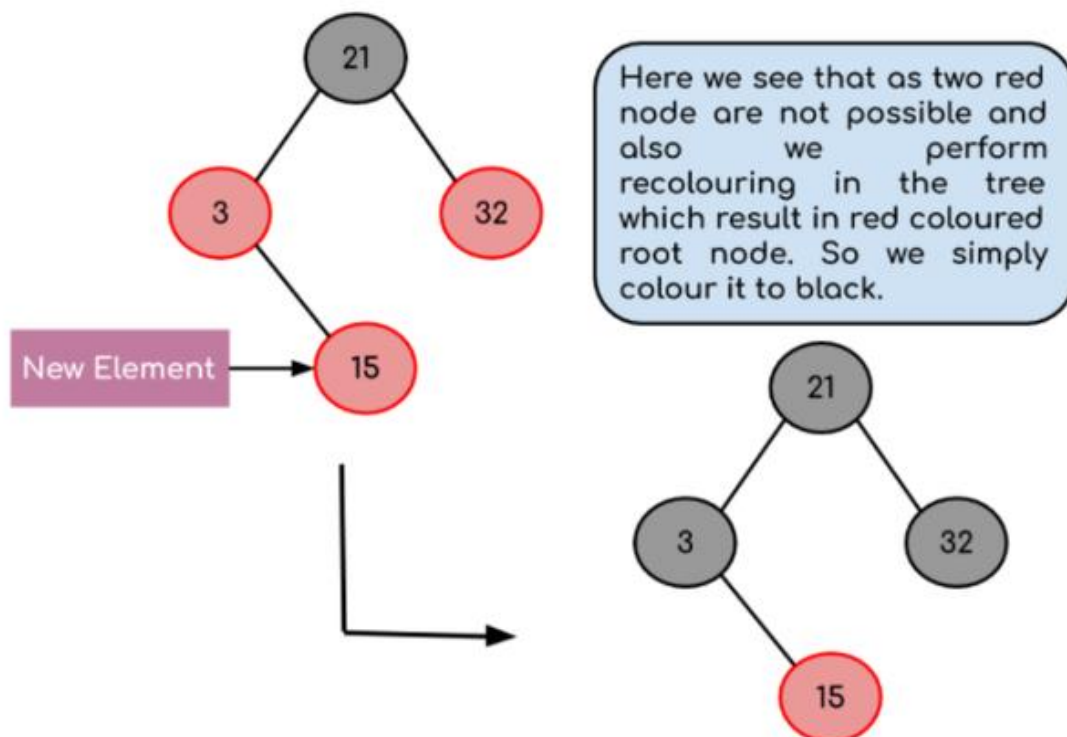
Step 2: Inserting element 21 inside the tree.



Step 3: Inserting element 32 inside the tree.



Step 4: Inserting element 15 inside the tree.



Final Red Black Tree:

